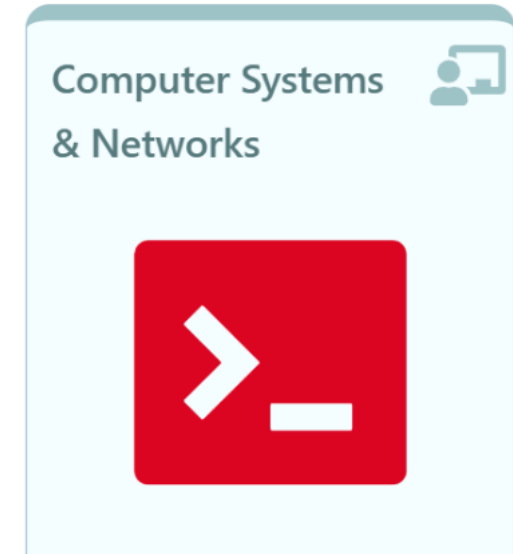


Computer Systems & Networks



Week 2 Class 1 LAB SOLUTIONS

From your **01. Wildcards** lab

- Using `find` to return files that begin with **temp**

```
compsys@compsys-virtualbox:~$ find ~ -name "temp*"
/home/compsys/temp
/home/compsys/snap/firefox/common/.mozilla/firefox/o49vw619.default/storage/temporary
/home/compsys/tempdate
compsys@compsys-virtualbox:~$
```

File Manipulation

01. Wildcards

02. Copy Move Remove

03. IO Redirection

Cisco NetAcad

EXERCISE: Using `find`

Write the command to find and return all files whose name begins with **tmp**

- The **find** command is a file search utility (Search for files in a directory hierarchy).

```
$ find ./Desktop -name "comp*" #command
./Desktop/computer.desktop #output
```

- Sample output:

```
compsys@compsys-virtualbox:~$ find . -name "tmp*"
./tmp
./snap/firefox/common/.mozilla/firefox/o49vw619.default/datareporting/glean/tmp
compsys@compsys-virtualbox:~$
```

Week 2 Class 1 LAB SOLUTIONS

From your **03. IO Redirection** lab

File Manipulation

01. Wildcards

02. Copy Move Remove

03. IO Redirection

Cisco NetAcad

- **ls /usr/bin** is the command run
- **> command.txt** redirects the output of the **ls** command into the text file
- **2>&1** sends the output of the FD2 (stderr) to the same location as the FD1 (stdout)

This could be re-written more streamlined as

```
$ls /usr/bin ____ command.txt <<fill in the blank>>
```



EXERCISE:

Usign redirection

- allow the user to add (**echo**) a new record to the **employee.txt** file

Ensure this new record is appended to the existing **employee.txt** file, test it

```
compsys@compsys-virtualbox:~$ echo "123 Caroline Manager Sales €1234" >> employee.txt
compsys@compsys-virtualbox:~$ cat emp*
300 frank manager sales €5000
400 caroline developer technology €5500
999 catherine dev tech €4500
100 mary sysadmin technology €7000
200 rosanne manager marketing €9500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
echo "There are no $ currency, only € in employee.txt"
just another example katherine
666 Katherine dev tech €6000
123 Caroline Manager Sales €1234
```

01: Search and Filter



GREP

sort · cut · grep

02: Regular Expressions



basic RegEx · extended RegEx · metacharacters

03: Positional Parameters



\$0 \$1 \$2
cp file1.txt file2.txt
\$@

Positional Parameters · exit

Lab



```
#!/bin/bash
```

grep · RegEx · Variables · Positional Parameters · Exit

Sorting Files or Input

↓ Z ↓ A
↓ A ↓ Z

↓ 9 ↓ 1
↓ 1 ↓ 9

Sorting Files for Input

- The **sort** command can be used to rearrange the lines of files or input in either dictionary or numeric order.

\$ sort filename

- **sort** command can rearrange the output based on the contents of one or more fields

\$ sort employee.txt

Delimited Files

- Fields are determined by a *field delimiter* contained on each line
 - **tab** character is the default delimiter
 - REMEMBER a "space" in between each word is actually a *single tab character*

Fields and Sort Options

- the following command can be used to **sort** the `passwd` file (alphabetically by first column by default)

```
compsys@compsys-virtualbox:~$ sort -t: passwd
```

- **-t** option will allow for another field separator to be specified
- You can specify to sort by a specific field (i.e. column 3) in numericcal order:

```
lbox:~$ sort -t: -n -k3 passwd
```

- **-n** option is used to perform a numeric sort.
- **-k** option allows you specify which field to **sort** by

Viewing File Statistics

109

Lines

1518

Words

72

Sentence

9336

Characters

Viewing File Statistics

- The **wc** command provides the number of lines, words and bytes (1 byte = 1 character in a text file) for a file, and a total line count if more than one file is specified.

```
$ wc employee.txt
```

- It is also possible to view only specific statistics:
 - Use the **-l** option to show just the **number of lines**
 - The **-w** option to show just the **number of words**
 - The **-c** option to show just the **number of bytes**
 - or any combination of these options

Command Line Pipes

The **pipe** | character → used to send the output of one command to another.

E.g. The pipe character allows you to utilise the head and tail commands not only on files, but on the output of other commands.

Multiple pipes can be used consecutively to **link** multiple commands together. Each command only sees input from the previous command.

Filtering Files

```
ubuntu@golinux:~$ cat fruits.txt
apple
banana
mango
orange
watermelon
ubuntu@golinux:~$ cut -b 1 fruits.txt
a
b
m
o
w
```

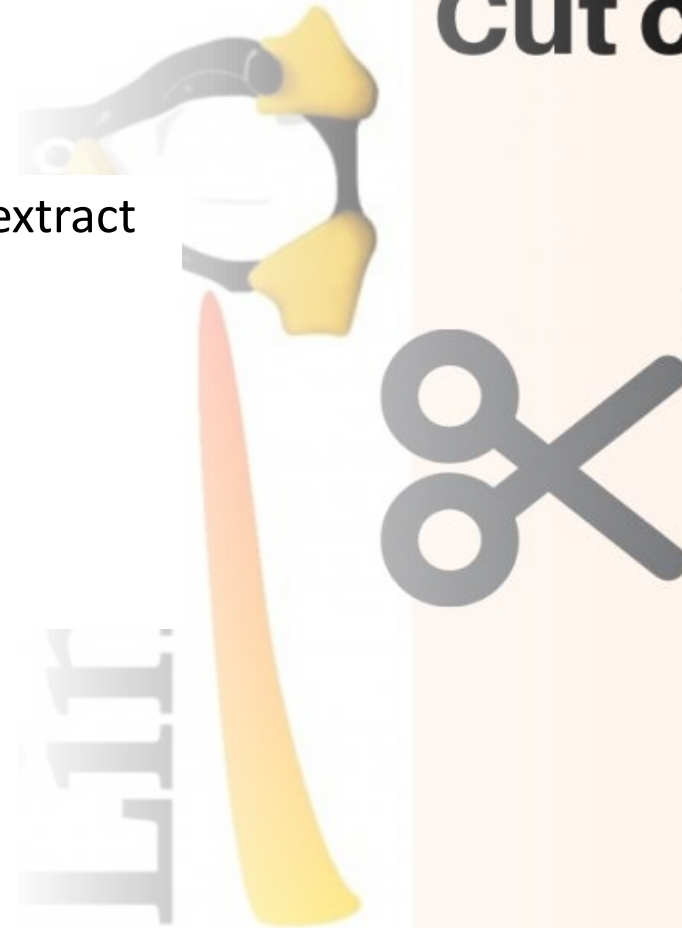

Filter File Sections

The **cut** command can extract *columns* of text from

- i. a file or
- ii. standard input.

Cut command in Linux

In Linux the cut command is used to snip a portion from a file or we can say to print a particular section or a particular line. This command first appeared in AT&T System III UNIX in 1982.



Filter File Sections

- The **cut** command is for working with delimited files

```
albox:~$ cut -d: -f1,5-7 passwd
```

- **-d** option allows a specific alternative *delimiter* (default in **cut** expects input to be separated by the **Tab/space** character)
- **-f** option can specify *which fields to display* (here it displays the first, fifth, sixth and seventh fields from a `passwd` file)

```
albox:~$ cut -d: -f1,5-7 passwd | head -3
```

Filter File Sections

-c option is used to extract columns of text based upon character position

Return only the first three characters of the *employee.txt* file:

```
oX:~$ cut -c1-3 empl*
```




grep

(globally search
for a regular
expression and
print matching
lines)

grep

on every distribution
of Linux

search directories and
files for keywords and
strings

- search input files for a search string, and print the lines that match it

is a pattern searcher

- If the pattern is found, it displays the line containing it.

\$ grep *string filename*

grep

One of the renowned
search tool on Unix-
like systems

can be used to search
for anything

- a file
- or a line
- or multiple lines

\$ grep *string filename*

grep

Versions

grep
#basic

grep -E
#extended



grep

(globally search
for a regular
expression and
print matching
lines)

What is regex?

Regex (Regular Expression) are special symbols used to build search patterns

Imagine you want to find certain words or patterns inside files.

- Regex lets you describe **rules** for what to search for, instead of typing exact words

When you **write a regex**, you're basically **combining these characters** to create a pattern the computer can follow

Syntax

```
compsys@compsys-virtualbox:~$ grep "sysadmin" employee.txt
100 mary sysadmin technology €7000
600 colm sysadmin technology €4500
```

- `-n` parameter to show line numbers

```
3:100 mary sysadmin technology €7000
6:600 colm sysadmin technology €4500
```

- `-v` prints all of the lines that do not match the search string

```
1:300 frank manager sales €5000
2:400 caroline developer technology €5500
4:200 rosanne manager marketing €9500
5:500 eamonn dba technology €6000
```


Syntax

- **-c** option tells grep to suppress the printing of matching lines, and only display the number of lines that match (or **-v** for *do not* match) the query.

```
compsys@compsys-virtualbox:~$ grep -c "sysadmin" employee.txt
2
compsys@compsys-virtualbox:~$ grep -cv "sysadmin" employee.txt
4
```

Using basic regex metacharacters with `grep`

+ Equivalent to one or more occurrences of previous character.

? almost 1 repetition of previous character (**a?** match 'a' or 'aa')

(Start of alternation expression

) End of alternation expression

| Matching either of the expression separated by '|'.

- **"(a|b)cde"** would match either **'acde'** or **'bcde'**

{ start of range specifier.

- **"a{2}"** matches **"aa"** in file i.e. the letter 'a' 2 times.

} end of range specifier

Using basic regex metacharacters with `grep`

`^` Start of line

`$` End of line

`.` Any single character

- `grep "c.t" employee.txt` → matches `cat`, `cut`, `cot`

`\.` Literal dot

- `grep "\.txt" employee.txt` → matches `.txt`
- In basic regex, you **escape special chars** to match them literally (`.` `*` `[ ]`)

`\+` One or more of previous character

- `grep "lo\+" employee.txt` → matches `lo`, `loo`, `looo` (needs `\` in basic `grep`)

`[abc]` Any one of `a`, `b`, or `c`

`[a-m]` Range of characters

Using basic regex metacharacters with **grep**

* = regex means search for zero or more instances of preceding character

```
$ grep 'ar*' employ*
```

PART 1: 'ar*' matches "a" (zero rs), "ar" (one r), "arr" (two rs), "arrr" (three rs), and so on.

PART 2: employ* is **shell globbing**, not part of grep's regex.

In the shell:

- * matches zero or more characters.
- So employ* matches **all files starting with employ**, like employee.txt, employment.csv, employer.log, etc.

So grep is actually searching inside all files whose names start with "employ".

Using basic regex metacharacters with **grep**

^ → caret metacharacter indicated the beginning of the line

```
compsys@compsys-virtualbox:~$ grep '^4' employ*  
400 caroline developer technology €5500
```

\$ metacharacter indicates the end of the line

```
y €4500  
~$ grep '4$' employ*
```

So then, what will this search for?

```
$ grep '^4$' employ*
```


grep as a filter

- as a filter with other commands
- “|” pipe the output of the command through grep.

```
~$ ls -l | grep '^d'
```

Tips: use `grep -E`

Most “special” characters like `+` `?` `|` `()` `{}` need a **backslash** `\` in **basic grep**.

- Anchors (`^` and `$`) and character classes (`[]`) **work without -E**.
- If you want to avoid backslashes:

In general, use `grep -E` (extended regex)

Using metacharacters **grep -E**

grep -E lets you use more powerful search patterns.

The -E means Extended Regular Expressions

So you can use:

- **OR (|)** to search for *this OR that*:

```
grep -E "sysadmin|engineer" employee.txt
```

- **Grouping with parentheses ()**

```
grep -E "(error|warning|failed)" errors.txt
```

If your pattern includes |, (), +, or ? → use grep -E

♥ grep ♥

JULIA EVANS
@b0rk

grep lets you search
files for text

```
$ grep bananas foo.txt
```

Here are some of my
favourite grep command
line arguments!

 **-i** case insensitive

 **-A** Show context for your
search.

 **-B**  **-C**
\$ grep -A 3 foo
will show 3 lines of
context after a match

 **-E**
aka
egrep

Use if you want
regexps like ".+" to
work. otherwise you
need to use ".\+"

 **-v**

invert match: find
all lines that don't
match

 **-l**

only show the
filenames of the files
that matched

 **-F**

don't treat the match
string as a regex
eg \$ grep -F ...

 **-r**

recursive! Search
all the files in
a directory.

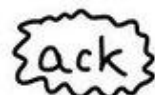
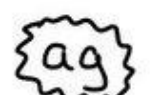
 **-o**

only print the
matching part of
the line (not the
whole line)

 **-a**

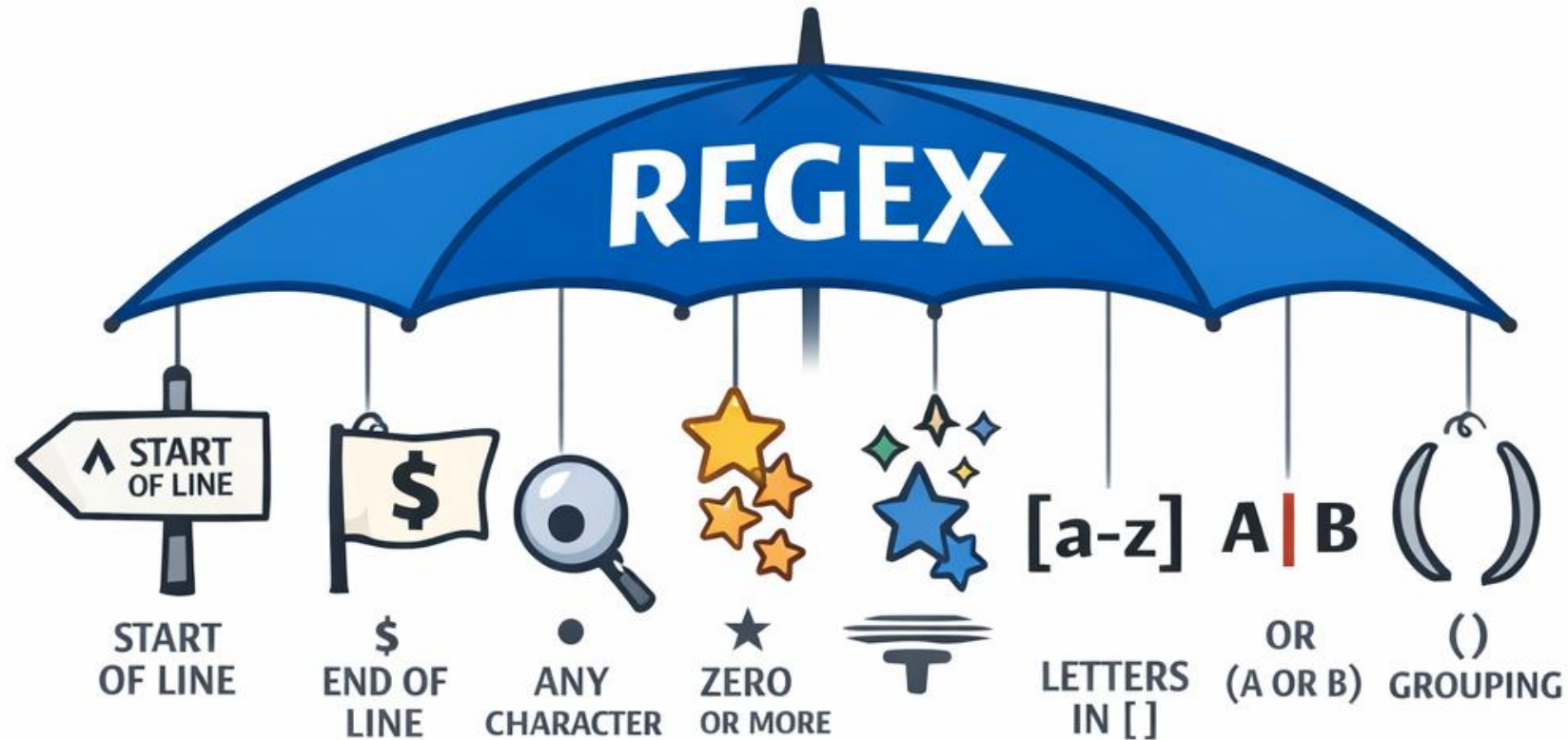
search binaries:
treat binary data
like it's text instead
of ignoring it!

grep alternatives

 ack  ag  ripgrep
(better for searching code!)

Regex = The Umbrella Term

Pattern matching language



Special Characters (Regex Tools)

Special symbols used to build search patterns

Try out this weeks Lab

01: Search and Filter



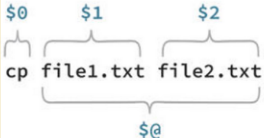
sort · cut · grep

02: Regular Expressions



basic RegEx · extended RegEx · metacharacters

03: Positional Parameters



Positional Parameters · exit

Lab



grep · RegEx · Variables · Positional Parameters · Exit

#!/bin/bash

Lab

Computer Systems & Networks

Current Week 2

Search

Computer Syste...

02: File and T...

02: Filtering ...

Lab

EXERCISES:

- Assume you wish to write a script called **numEmp**

```
compsys@compsys-virtualbox:~$ cat numEmp*  
echo "Today on `date` the total number of employees are"  
cat ~/employee.txt | wc -l
```

How could you improve on this to:

- Match all lines that start with **cat**
- Match all lines that end with **cat**
- Match all lines that contain any of the letters 'a', 'b' or 'c'
- Match all lines that do not contain a vowel
- Match all lines that start with a digit following zero or more spaces
- Match all lines that contain the word hello in upper-case or lower-case
- \$ **grep "\$*" file1** will match the lines that contain _____ at the _____ of